

Contents

Guide fil rouge – Mile stone 6	1
--	---

Guide fil rouge – Mile stone 6

Le support est propriété de **Bechir Bejaoui**

MiniStock – Papeterie Express

(après le chapitre 12 – Modules et Packages)

Objectif

Transformer le programme en un **package Python modulaire** professionnel :

- organisation en plusieurs fichiers
- imports relatifs
- point d'entrée via `__main__`
- lancement avec `python -m ministock`
- structure prête pour des évolutions futures (tests, configuration, etc.)

1. Structure finale du projet

Remplacez le contenu du dossier `projet_fil_rouge/` par cette arborescence :

```
projet_fil_rouge/  
  README.md  
  requirements.txt  
  ministock/  
    __init__.py  
    data.py  
    operations.py  
    affichage.py  
    main.py
```

2. Contenu des fichiers

`ministock/init.py`

```
# ministock/__init__.py  
# Package MiniStock – Papeterie Express  
  
from .data import stock, produits, SEUIL_ALERTE, TAUX_TVA  
from .operations import ajouter_article, ajouter_stock, vendre  
from .affichage import afficher_inventaire, lister_critiques
```

```

__all__ = [
    "ajouter_article", "ajouter_stock", "vendre",
    "afficher_inventaire", "lister_critiques",
    "stock", "produits", "SEUIL_ALERTE", "TAUX_TVA"
]

```

ministock/data.py

```

# ministock/data.py
# Données et constantes globales

```

```

SEUIL_ALERTE = 5
TAUX_TVA = 0.19

```

```

stock = {
    "A001": 18,      # Cahiers 96 pages
    "B012": 3,
    "C005": 32,
    "D008": 0,
    "E015": 12
}

```

```

produits = [
    {"ref": "A001", "nom": "Cahier 96 pages", "prix_ht": 3.50},
    {"ref": "B012", "nom": "Stylo bleu", "prix_ht": 1.20},
    {"ref": "C005", "nom": "Ramette A4", "prix_ht": 4.99},
    {"ref": "D008", "nom": "Marqueur noir", "prix_ht": 2.80},
    {"ref": "E015", "nom": "Gomme blanche", "prix_ht": 0.80}
]

```

ministock/operations.py

```

# ministock/operations.py
# Fonctions métier

```

```

from .data import stock, produits, TAUX_TVA

```

```

def ajouter_article(ref: str, nom: str, prix_ht: float, quantite_initiale: int = 0) -> bool:
    """Ajoute un nouvel article au catalogue et au stock."""
    if any(p["ref"] == ref for p in produits):
        print(f"Erreur : la référence {ref} existe déjà.")
        return False

    produits.append({"ref": ref, "nom": nom, "prix_ht": prix_ht})
    stock[ref] = quantite_initiale
    print(f"Article ajouté : {nom} (réf {ref}) - {quantite_initiale} unités")
    return True

```

```

def ajouter_stock(ref: str, quantite: int) -> bool:
    """Réapprovisionne un article existant."""
    if ref not in stock:
        print(f"Erreur : référence {ref} inconnue.")
        return False

    stock[ref] += quantite
    print(f"Réapprovisionnement : +{quantite} unités pour réf {ref}")
    print(f"Nouveau stock : {stock[ref]} unités")
    return True

def vendre(ref: str, quantite: int) -> bool:
    """Enregistre une vente si le stock est suffisant."""
    if ref not in stock:
        print(f"Erreur : référence {ref} inconnue.")
        return False

    if stock[ref] < quantite:
        print(f"Erreur : stock insuffisant pour réf {ref} (disponible : {stock[ref]})")
        return False

    stock[ref] -= quantite
    prix_ht = next(p["prix_ht"] for p in produits if p["ref"] == ref)
    total_ht = quantite * prix_ht
    total_ttc = total_ht * (1 + TAUX_TVA)

    print(f"Vente enregistrée : {quantite} × réf {ref}")
    print(f"Total HT : {total_ht:.2f} TND")
    print(f"Total TTC : {total_ttc:.2f} TND")
    print(f"Stock restant : {stock[ref]} unités")
    return True

```

ministock/affichage.py

```

# ministock/affichage.py
# Fonctions d'affichage

```

```

from .data import stock, produits, SEUILALERTE, TAUX_TVA

```

```

def afficher_inventaire():
    """Affiche l'inventaire complet avec valeur et statuts."""
    print("\nInventaire - MiniStock")
    print("-" * 80)

```

```

print(f"{'Réf':<6} {'Produit':<24} {'Prix HT':>10} {'Qté':>6} {'Valeur HT':>12} {'Statut':>10}")
print("-" * 80)

valeur_totale_ht = 0
ruptures = 0
alertes = 0

for p in produits:
    ref = p["ref"]
    qte = stock.get(ref, 0)
    valeur = qte * p["prix_ht"]
    valeur_totale_ht += valeur

    if qte == 0:
        statut = "RUPTURE"
        ruptures += 1
    elif qte <= SEUILALERTE:
        statut = "STOCK BAS"
        alertes += 1
    else:
        statut = "OK"

    print(f"{'ref':<6} {p['nom']:<24} {p['prix_ht']:>10.2f} {qte:>6} {valeur:>12.2f} {statut:>10}")

print("-" * 80)
print(f"Valeur totale stock HT : {valeur_totale_ht:>12.2f} TND")
print(f"Valeur totale stock TTC : {valeur_totale_ht * (1 + TAUX_TVA):>12.2f} TND")
print(f"Ruptures : {ruptures} | Stocks bas : {alertes}")
print("-" * 80)

def lister_critiques():
    """Liste les produits en rupture ou en stock bas."""
    print("\nProduits critiques à réapprovisionner :")
    print("-" * 50)
    critiques = False
    for p in produits:
        qte = stock.get(p["ref"], 0)
        if qte <= SEUILALERTE:
            print(f"• {p['nom']} (réf {p['ref']}) → {qte} unités")
            critiques = True
    if not critiques:
        print("Aucun produit critique pour le moment.")
    print("-" * 50)

```

ministock/main.py

```

# ministock/main.py
# Point d'entrée du programme

from .affichage import afficher_inventaire, lister_critiques
from .operations import ajouter_article, ajouter_stock, vendre
from .data import stock

def main():
    print("=====")
    print("  MiniStock - Papeterie Express")
    print("  Version finale - Package modulaire")
    print("=====\n")

    afficher_inventaire()
    lister_critiques()

    # Démonstrations
    print("\nDémonstrations :")
    ajouter_article("F022", "Crayon HB", 0.60, 80)
    ajouter_stock("B012", 20)
    vendre("A001", 5)
    vendre("D008", 1)          # devrait échouer

    print("\nInventaire après opérations :")
    afficher_inventaire()

if __name__ == "__main__":
    main()

```

3. Exécution du package

```

cd projet_fil_rouge
# Activer l'environnement virtuel si ce n'est pas déjà fait
# Linux/macOS :
source .venv/bin/activate
# Windows :
# .venv\Scripts\activate

# Lancement du programme
python -m ministock

```